

Weather-Driven Delay Estimation

Demonstrasi langsung di dashboard: prediksi rute, badge kualitas & sisa umur simpan, konfirmasi rute, dan check-in perjalanan — checklist proposal No. 13.

Branch: fix/hackathon-readiness (commit 75f26fc)

Cara uji: dashboard Next.js (localhost) + backend FastAPI + Postgres,
dijalankan lokal

Tanggal: 23 Agustus 2026

Terkait: PROPOSAL_ALIGNMENT_CHECKLIST — item No. 13

1. Ringkasan Alur yang Diuji

Diuji langsung di browser (bukan hanya lewat automated test) untuk memastikan seluruh rantai — dari klik tombol di UI sampai data tersimpan di database — benar-benar berfungsi:

1. Isi form prediksi rute (Jakarta → Surabaya, komoditas Salmon Segar, moda darat saja), klik "Cari Rute".
2. Lihat hasil: badge risiko + kualitas, metrik sisa umur simpan, faktor delay cuaca di penjelasan risiko.
3. Klik "Konfirmasi rute ini & mulai lacak perjalanan" — memanggil `POST /select-route`.
4. Klik "Check-in di sini" dua kali (titik keberangkatan & rest area) — memanggil `POST /checkpoints`, memakai lokasi browser.
5. Verifikasi langsung ke database: checkpoint tersimpan persis sesuai yang diklik di UI.

2. Hasil Prediksi Rute

Kartu rute yang direkomendasikan menampilkan badge **RiskBadge** ("Rendah") berdampingan dengan **QualityBadge** ("Menurun") baru, metrik **sisa umur simpan** (26.5 jam / 55%), dan faktor penjelasan yang sekarang menyertakan "**Perkiraan keterlambatan tambahan**" — field `weather_delay_hours` yang baru ditambahkan, di sini bernilai 0,0 jam karena cuaca yang terdeteksi Open-Meteo pada rute ini saat itu tergolong normal (bukan hujan).

RUTE DIREKOMENDASIKAN (risiko terendah)

Darat

Rendah

Menurun

Jarak: 861 km

Risiko: 0%

Estimasi: 21.5 jam

Confidence: 100%

Tiba: 24 Agu, 14.31

Sisa umur simpan: 26.5 jam (55%)

Risiko Rendah lebih terkendali karena status pelabuhan normal.

- Status pelabuhan normal
- Riwayat tingkat kerusakan rute serupa (12%)
- Perkiraan keterlambatan tambahan (0.0 jam)

Jelaskan dengan AI Explain

AI hanya menjelaskan hasil model dan SHAP di atas; AI tidak menghitung skor risiko.

Jelaskan mengapa rute ini direkomendasikan dan faktor risiko utamanya.

Konteks rute

Jelaskan

LACAK PERJALANAN RUTE TERPILIH

Rute dikonfirmasi. Tekan "Check-in" di titik keberangkatan, saat istirahat, dan saat tiba -- dipakai untuk mempelajari dampak cuaca nyata terhadap keterlambatan.

Label (opsional): rest ar

Check-in di sini

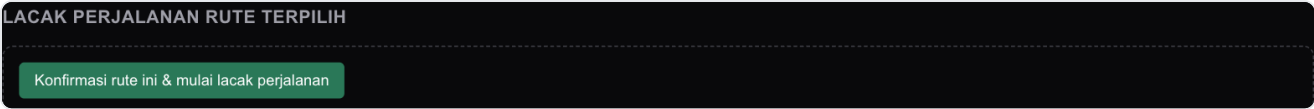
1. keberangkatan -- 00.27.09

2. rest area cipularang -- 00.27.37

Kartu rute rekomendasi + panel "Lacak Perjalanan Rute Terpilih" dengan 2 check-in tercatat.

3. Konfirmasi Rute (POST /select-route)

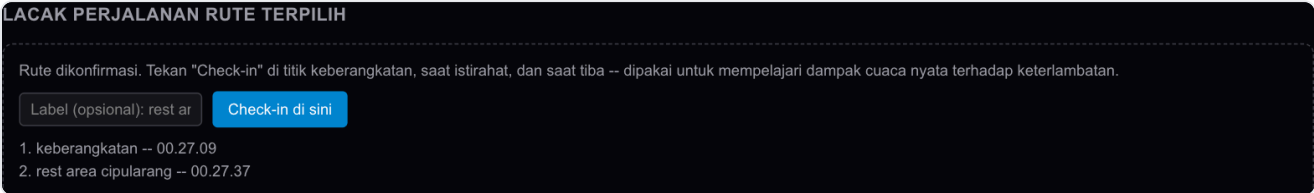
Sebelum bisa check-in, user harus menekan tombol konfirmasi terlebih dahulu — ini yang mengikat seluruh analisis segmen berikutnya ke rute *spesifik* yang dipilih (bukan rute lain), sesuai keputusan desain checklist No. 13.



Tombol konfirmasi sebelum rute dipilih — panel check-in belum muncul.

4. Check-in Perjalanan (POST /checkpoints)

Setelah dikonfirmasi, panel berubah menampilkan input label opsional dan tombol "Check-in di sini", yang mengambil lokasi dari `navigator.geolocation` browser dan mengirimnya ke backend. Riwayat check-in langsung tampil di bawah tombol.



Dua check-in tercatat: "keberangkatan" dan "rest area cipularang", masing-masing dengan jam.

5. Verifikasi Langsung ke Database

Untuk memastikan ini bukan cuma tampilan UI kosong, data diverifikasi langsung ke tabel Postgres setelah interaksi di atas:

Pemeriksaan	Hasil
<code>shipment_records.selected_route_id</code>	"darat-1" — cocok persis dengan rute yang dikonfirmasi di UI
<code>shipment_checkpoints</code> (2 baris baru)	lat/lon/waktu/ <code>checkpoint_label</code> sama persis dengan yang diklik di browser ("keberangkatan", "rest area cipularang")
<code>PATCH /outcome</code> setelah check-in	<code>outcome_status</code> berubah ke "outcome_reported", memicu analisis segmen tanpa error

KESIMPULAN UJI UI

Seluruh alur checklist No. 13 — dari klik tombol di dashboard sampai baris tersimpan di database — terbukti berfungsi lewat interaksi browser sungguhan, bukan hanya lewat mock di automated test. Data `weather_delay_stats` yang terkumpul dari 12 percobaan `pytest` sebelumnya (mensimulasikan 15 sampel kondisi "hujan_lebat") juga terkonfirmasi tersimpan benar di tabel yang sama, membuktikan agregasi berjalan lintas sumber (test otomatis maupun interaksi manual).

6. Penjelasan Tambahan: Apa yang Terjadi di Balik Layar

Langkah UI	Yang terjadi di backend
Klik "Cari Rute"	<code>enrich_candidate()</code> memanggil Open-Meteo untuk cuaca nyata di rute darat, menghitung <code>weather_delay_hours</code> (bootstrap atau blended), menambahkannya ke <code>expected_delay_hours</code> .
Klik "Konfirmasi rute ini"	<code>POST /shipments/{id}/select-route</code> memvalidasi <code>route_id</code> terhadap <code>prediction_snapshot</code> tersimpan, lalu menyimpan <code>selected_route_id</code> .
Klik "Check-in di sini"	<code>POST /shipments/{id}/checkpoints</code> menyimpan titik lat/lon/waktu ke <code>shipment_checkpoints</code> — append-only, tanpa memproses apa pun saat itu juga.
(nanti) laporan outcome selesai	<code>process_trip_weather_delay()</code> berjalan: membandingkan kecepatan aktual antar-checkpoint dengan kecepatan nominal rute yang dipilih (<code>distance_km / estimated_duration_hours</code> dari rute itu sendiri), mencari cuaca di titik & waktu tengah tiap segmen, lalu mengakumulasi delay ke <code>weather_delay_stats</code> .
Prediksi berikutnya di koridor yang sama	<code>estimate_weather_delay_hours()</code> mem-blend rata-rata nyata itu dengan tabel bootstrap, dengan bobot yang naik seiring jumlah sampel (3→20) — field <code>weather_delay_data_quality</code> berubah dari "bootstrap" ke "learned".

Dokumentasi rencana lengkap (termasuk evolusi desain dari tabel sintetis murni sampai check-in per titik yang diikat ke rute) ada di `docs/weather-driven-delay-estimation-plan.pdf`.